

AI Repository Architect

Automated Repository Documentation Report

Repository ID: 7fb3652c-a13b-4a45-be13-c971b2d17bdd

Generated using AI-powered code analysis

Project Overview

This project is designed to generate dynamic and dramatic narratives for a fantasy RPG duel represented as a game of chess. Each chess move corresponds to an action in the battle, and the system provides a narrative sentence and a matching visual spell effect for each move. The core functionality relies on a locally-hosted, open-source Language Model (LLM) through Ollama, which generates the narrative and effect based on the move data provided.

Architecture

The architecture is centered around a single function `get_narration`` that interacts with the LLM to generate narratives and effects. It uses helper functions `_fallback_narration`` and `_fallback_effect`` to provide default responses in case the LLM fails to generate valid output. The system is designed to be robust, ensuring that the game can continue even if the LLM is unreachable or returns unparsable data.

- **Input**: Chess move data in JSON format.
- **Processing**: The move data is sent to the LLM to generate a narrative and effect. If the LLM fails, fallback mechanisms are used.
- **Output**: A JSON object containing the narrative sentence, the visual effect, and the source of the response (either the LLM or the fallback mechanism).

Folder Structure

The repository does not contain multiple folders; it consists primarily of a single Python script. However, the following components are defined within the script:

- `build_prompt``: Function to create the prompt for the LLM.
- `_fallback_narration``: Function to generate a fallback narrative sentence.
- `_category``: Helper function to categorize the move.
- `get_narration``: Main function to generate the narrative and effect.
- `_fallback_effect``: Function to select a fallback visual effect.

Key Components

- **LLM Interaction**: The system talks to a locally-hosted LLM through Ollama to generate dynamic content.
- **Fallback Mechanisms**: Ensures that the game can continue with predefined narratives and effects if the LLM fails.
- **Data Handling**: Processes JSON input representing chess moves and outputs structured JSON with the narrative and effect.

Main Functions

- `build_prompt(move_info)`: Constructs a prompt string for the LLM based on the move information.
- `_fallback_narration(move_info)`: Provides a default narrative sentence if the LLM cannot generate one.
- `_category(move_info)`: Determines the category of the move (e.g., checkmate, capture).
- `get_narration(move_info, timeout=8)`: The main function that handles the interaction with the LLM, processes the response, and applies fallback mechanisms if necessary.
- `_fallback_effect(move_info)`: Selects a default visual effect from a predefined list based on the move category.

API Endpoints

There are no explicit API endpoints in this repository. The interaction with the LLM is handled through HTTP POST requests to a locally-hosted service at `OLLAMA_URL`.

Data Flow

1. **Chess Move Data**: The system receives JSON data representing a chess move.

2. **Prompt Construction**: The `build_prompt` function constructs a prompt for the LLM.
3. **LLM Interaction**: The `get_narration` function sends the prompt to the LLM and waits for a response.
4. **Response Parsing**: The response from the LLM is parsed to extract the narrative and effect.
5. **Fallback Application**: If the LLM response is invalid or the LLM is unreachable, fallback mechanisms (`_fallback_narration` and `_fallback_effect`) are applied.
6. **Output Generation**: The system outputs a JSON object containing the narrative, effect, and source of the response.

Setup Instructions

1. **Install Dependencies**: Ensure you have Python installed along with the `requests` library. You can install the library using pip:

```
```bash  

pip install requests
```
```

2. **Configure Environment Variables**: Set the `OLLAMA_URL`, `MODEL_NAME`, and other necessary environment variables to point to your locally-hosted LLM.
3. **Run the Script**: Use the script in your application by calling the `get_narration` function with the appropriate chess move data.

Future Improvements

- **Enhanced Fallbacks**: Improve the fallback mechanisms to provide more varied and contextually relevant responses.
- **Error Handling**: Add more detailed logging and error handling to diagnose issues with the LLM interaction.
- **User Interface Integration**: Develop a user interface to display the generated narratives and visual effects in real-time during the chess game.
- **Model Training**: Consider training a custom model specifically for generating fantasy RPG narratives to improve the quality and relevance of the output.

- **Performance Optimization**: Optimize the data flow and processing to handle high-frequency requests efficiently, especially in a live gaming scenario.